

Revisión cruzada de validaciones y manejo de errores

Revisión del 29 de septiembre de 2026 sobre la rama del backend. Se cruzan tres fuentes que deben coincidir: el **frontend** (validación en cliente y manejo de errores en `frontend/src`), el **backend** (esquemas de entrada, dominio y `errors.py`) y el **contrato** (`docs/contrato-api-jsonapi.md`). El mock del frontend (`mockAdapter.js`) se usa como especificación ejecutable de lo que la interfaz espera.

Resumen

Resultado	Cantidad
Reglas revisadas (formularios, dominio, parámetros)	19
Códigos de error revisados	13
Hallazgos corregidos en esta revisión	5
Hallazgos documentados como pendientes	3
Pruebas nuevas	23 integración + 4 unitarias (suite total: 157, todas pasan)

1. Validaciones: frontend vs. backend

Campo / regla	Frontend (cliente)	Backend (fuente de verdad)	¿Coinciden?
Adoptante · nombre	≥ 2 caracteres (recortado)	2–80, recortado	✓ (el backend además limita el máximo)
Adoptante · email	regex <code>^[^\s@]+@[^\s@]+\.[^\s@]+\$</code>	misma regex; único sin distinguir mayúsculas	✓
Adoptante · teléfono	<code>^\d{7,13}\$</code>	misma regex, tras recortar espacios de los extremos	✓
Adoptante · ciudad	≥ 2 caracteres	≥ 2, recortado	✓
Postulación · mensaje	10–500 (recortado), <code>maxlength=500</code>	10–500 en el borde y en la entidad <code>Postulacion</code>	✓
Animal · nombre	obligatorio, <code>maxlength=60</code>	1–60, recortado	✓
Animal · edad	entero ≥ 0	entero ≥ 0 (borde y entidad)	✓
Animal · especie / sexo	<code><select></code> con valores fijos	enums <code>Especie / Sexo</code>	✓
Animal · foto	<code>type="url"</code> (solo el navegador)	antes: cualquier texto → ahora <code>http(s)://</code> o <code>data:image/</code>	🔧 corregido

Campo / regla	Frontend (cliente)	Backend (fuente de verdad)	¿Coinciden?
Animal · temperamento	texto separado por comas	antes: sin tope → ahora ≤ 15 rasgos de ≤ 40 caracteres	 corregido
Animal · detalles de especie	según especie	antes: sin tope → ahora ≤ 20 claves	 corregido
Animal · estado	no se envía	se ignora si llega; siempre nace disponible	✓
Resolver postulación · estado	botones <i>Aprobar</i> / <i>Rechazar</i>	aprobada o rechazada ; otro valor → 422	✓
Regla 1 · animal no disponible	la UI deshabilita "Postular" si está adoptado	dominio: 409 ANIMAL_NO_DISPONIBLE	✓
Regla 2 · postulación duplicada	— (depende del backend)	caso de uso: 409 POSTULACION_DUPLICADA	✓
Regla 3 · cierre automático	la UI avisa cuántas se cerrarán	caso de uso AprobarPostulacion	✓
page[size]	el panel pide 50	máximo 50 → 400	✓
Filtros filter[...]	valores de los <select>	enums → 400 con source.parameter	✓
include	refugio / animal, adoptante	lista permitida → 400 si no	✓

Paridad mock ↔ API real: las reglas del `mockAdapter.js` (validaciones del formulario, reglas 1–3, reutilizar adoptante por email, 404 de adoptante inexistente, 422 para un estado inválido) coinciden con el backend. Los mensajes de las reglas son idénticos ("Luna ya fue adoptada.").

2. Códigos de error: backend vs. interfaz

Código	HTTP	¿Lo emite el backend?	¿Qué hace la interfaz?
VALIDACION	422	✓ un error por campo con <code>source.pointer</code>	Marca el campo exacto (<code>fieldErrors()</code>)
ANIMAL_NO_DISPONIBLE	409	✓	Mensaje propio y recarga la ficha
POSTULACION_DUPLICADA	409	✓	Mensaje propio
POSTULACION_YA_RESUELTA	409	✓	Mensaje propio y recarga el panel
RECURSO_NO_ENCONTRADO	404	✓	Mensaje propio
TRANSICION_INVALIDA	409	✓	antes: sin mensaje propio →  agregado
SOLICITUD_MALFORMADA	400	✓	antes: sin mensaje propio →  agregado

Código	HTTP	¿Lo emite el backend?	¿Qué hace la interfaz?
ERROR_INTERNO	500	✓ sin filtrar detalles	antes: sin mensaje propio →  agregado
CONTENT_TYPE_INVALIDO	415	✓	No aplica: el adaptador HTTP siempre envía el media type
ID_NO_COINCIDE	409	✓	No aplica: la interfaz arma el <code>id</code> de la URL y del cuerpo igual
METODO_NO_PERMITIDO / ERROR_HTTP	405 / otros	✓	Muestra el <code>title</code> (texto del backend)
REGLA_DE_NEGOCIO	409	✓ solo para reglas nuevas sin mapeo	Muestra el <code>detail</code> (texto del dominio)
RED (sin conexión)	—	— (lo genera el frontend)	"No se pudo conectar con el servidor"

Antes de esta revisión, los códigos sin mensaje propio no fallaban: se mostraba el `detail` en español que envía el backend. El cambio solo da un mensaje más claro y estable.

3. Hallazgos

Corregidos

- URL de foto sin validar (seguridad).** El backend aceptaba cualquier texto como foto, incluido `javascript:alert(1)`, y la interfaz lo pinta en `<img src>`. Ahora solo acepta `http(s)://` o `data:image/`, lo que cubre la semilla de Mongo y el mock ([schemas.py](#)).
- Campos libres sin tope de tamaño.** `temperamento`, `fotos` y `detallesEspecie` permitían documentos arbitrariamente grandes. Ahora tienen límites razonables, con mensaje en español por campo.
- Errores de campos lista no se marcaban en la interfaz.** Para `/data/attributes/fotos/0`, `fieldErrors()` tomaba `"0"` como nombre de campo. Ahora el frontend toma el segmento que sigue a `attributes` ([errors.js](#)), y el formulario de publicación muestra el error bajo "URL de foto".
- Errores duplicados y punteros con etiquetas internas.** Un campo que acepta dos formas (URL u objeto) producía dos errores con punteros como `/fotos/0/constrained-str`. Ahora el backend emite un error por campo con un puntero limpio ([errors.py](#), `_campo_y_ruta`).
- Códigos sin mensaje propio en la interfaz:** `TRANSICION_INVALIDA`, `SOLICITUD_MALFORMADA` y `ERROR_INTERNO`.

Pendientes (documentados)

- Carrera en la regla 2 con MongoDB.** Dos postulaciones idénticas simultáneas pueden pasar ambas `existe_pendiente()`. El índice único parcial `postulacion_pendiente_unica` detiene la segunda, pero el adaptador de Mongo (aún no implementado) debe traducir `DuplicateKeyError` a `PostulacionDuplicada`; si no, respondería 500.
- Registro con email existente ignora los datos nuevos.** Si un adoptante vuelve con el mismo email pero otro teléfono, `POST /adoptantes` devuelve el registro anterior sin avisar. Es la decisión del contrato (5.3), pero la interfaz podría indicarlo.

3. El `detail` de un 404 incluye el id pedido ("No existe el animal fantasma."). No es sensible en este sistema (ids públicos), pero conviene saberlo si se agregan recursos privados.
-

4. Evidencia

- Pruebas de esta revisión: `test_revision_validaciones.py` (URLs peligrosas, topes, valores límite, paridad del teléfono con el frontend) y `test_campo_y_puntero_ignoran_etiquetas_internas_de_pydantic` en `test_errores_http.py`.
- Suite completa del backend: **157 pruebas, todas pasan** (unitarias sin red + integración HTTP + integración con MongoDB real).
- El frontend compila con los cambios (`docker compose build frontend`).